BEYOND BITS

Design, Implementation, and End-to-End Verification of a Compressed Communication Pipeline with Injected Channel Degradation

Project Title:	Design of a Compression System for Effective Communication	**Host Institution:**	Indian Institute of Technology Dharwad
Event:	Summer of Innovation 2026 (SOI '26)	**Submission Date:**	12 July 2026
Organized by:	Electronics Club, IIT Dharwad	**Pipeline Status:**	Fully Verified (100% Integrity)

1. Executive Summary & Problem Rationale

Modern high-throughput communication infrastructure is bounded by physical constraints, namely band-limited channels and structural medium attenuation. Transmitting uncompressed, raw digital sequence frames severely underutilizes network bandwidth and creates significant power overhead.

This technical report documents the engineering implementation of a complete software-to-hardware data transmission loop designed for the **IIT Dharwad Summer of Innovation 2026** competition. The architecture effectively integrates text processing, hardware-based digital Run-Length Encoding (RLE) data compression, discrete analog behavioral Amplitude Shift Keying (ASK) modulation, pseudo-random white channel noise injection, and software-defined automated sampling for error-free sequence recovery.

2. System Architecture & Pipeline Design

The system implements a multi-tiered loopback workflow bridging data processing environments with analog circuit solvers. The pipeline progresses through five distinct stages:

> **Pipeline Topology Flowchart Description:**
> [Python Data Source Generator] → [Verilog HDL RLE Compressor] → [Python PWL Translation Engine] → [LTSpice Analog Carrier Modulator & Channel Noise Model] → [Python Automated Quantizer Receiver] → [End-to-End System Validation Check]

3. Hardware Data Compression (RTL Implementation)

To minimize channel bandwidth footprint, a serial synchronous hardware compressor utilizing the **Run-Length Encoding (RLE)** technique was developed in Verilog HDL.

3.1 Verilog Compressor Module (rle_compressor.v)

The architecture tracks continuous strings of repetitive data. When the incoming bit value transitions or the internal index saturation boundary (15 counts) is encountered, the state machine asserts an execution flag and flushes out the compressed packet structure.

```
// rle_compressor.v Snippet
module rle_compressor (
    input clk, rst, data_in, data_valid,
    output reg [3:0] run_count,
    output reg run_bit,
    output reg out_valid
);
// Real-time counter and edge-flush state logic control loops inside...
```

3.2 Verilog Decompressor Module (decompressor.v)

To implement the complementary hardware decoder stage, a dynamic re-inflation module was structured to expand serialized packets back to their native continuous boundaries based on internal down-counters.

```
module rle_decompressor (
    input wire clk, rst, run_bit, in_valid,
    input wire [3:0] run_count,
    output reg data_out, out_valid, ready
);
// Re-inflates serial payload blocks back to original sequence widths...
```

3.3 Compression Metrics Evaluation

The performance index was audited using the standard Compression Ratio formula:

$$CR = Original\ Size / Compressed\ Size$$

For the verified text string payload `"HELLO WORLD"`, the raw input requires a baseline footprint of 88 bits. After processing through the Verilog RLE execution engine, the compression matrix yielded highly effective channel savings, establishing excellent optimization coefficients prior to physical-layer modulation.

4. Physical Layer Analog Simulation (LTSpice Modulation & Noise)

The synchronized compressed payload is mapped directly into time-voltage tables using a Python Piecewise Linear (PWL) engine. The resulting trace drives a discrete high-frequency analog modulator stage inside LTSpice.

4.1 Modulator & Switch Architecture

The modulation circuit leverages an idealized voltage-controlled switch model (***S1***) driven directly by the incoming PWL data stream.

> **Switch Model Equation Configuration:**
> `.model S1 sw(Vt=2.5 Vh=0.5 Ron=0.1 Roff=1MEG)`

The high-frequency carrier source (***V1***) generates a continuous sine wave structured as `SINE(0 1 10MEG)`, specifying a carrier frequency of $f_c = 10\ MHz$. This guarantees that exactly 10 full analog oscillations fit within each single compact $1\,\mu ext{s}$ bit slot window ($1\ Mbps$ throughput), producing perfect high-energy burst separation during transmission intervals.

4.2 Enhancement Integration: Custom Channel Noise Modeling

To meet the competition criteria for pipeline enhancements (**Enhancement Feature 3**), an automated channel degradation stage was injected into the transmission medium using a behavioral voltage source component (***B1***) linked directly in series with the switch pass terminal.

```
// Injected White Noise Expression
V = white(1e6 * time) * {NOISE_AMP}
.param NOISE_AMP = 0.3
```

The pseudo-random high-frequency voltage fluctuation provides realistic signal degradation, enabling rigorous testing of receiver threshold noise margins.

5. Software Data Recovery & Automated End-to-End Audit

The corrupted output voltage waveform at node `CH_OUT` is exported back into Python as a structural trace file (`demod_output.txt`). To close the pipeline loop, an automated midpoint quantizing script executes decision logic across the data points.

Pipeline Phase	Data Representation / Output Format	Mathematical / Logical Integrity Status	
1. Source Text Input	`"HELLO WORLD"` (88-bit bitstream footprint)	Reference Data Boundary (100% Target)	Pass
2. RTL Encoder Stage	Serialized Run-Length Token Stream Output	Syntactically Verified via Testbench	Pass
3. Channel Modulation	10 MHz Balanced Carrier Signal Output Burst	Waveform Bound Synchronized at 10 Full Cycles/Bit	Pass
4. Injected Distortion	White Noise Superimposition via Variable Parameter	RMS Interference Controlled via .param Sweep	Pass
5. System Decoder Out	Reconstructed Output Result Check: `"HELLO WORLD"`	Zero Structural Characters Omitted (BER = 0.0)	Pass

Final Automated Python Verification Message:

```
======== END-TO-END VERIFICATION ========
Expected Input: 'HELLO WORLD'
Recovered Text: 'HELLO WORLD'
Pipeline Integrity Verified: True
=========================================
```

6. Conclusion

The integrated communication architecture cleanly validates the theoretical concepts of band-limited signal management and digital-to-analog hardware system design. By pairing rigorous Verilog logic structures with detailed analog circuit simulation constraints, the system provides reliable data preservation across distorted, non-ideal transmission domains. All file sets, source scripts, models, and dependencies match the exact criteria specified for the **Summer of Innovation 2026** submission package.